

is updated in the primary module, DRBD sends back the same block content to the secondary DRBD module in the peer node to replicate in its local disk, usually via a dedicated NIC. This way, DRBD keeps the data in both the nodes in sync. The primary node holds the service IP, i.e., the IP with which external applications communicate with the services provided. If the primary node fails, the secondary node takes over as the primary and the service IP by means of the HA service.

Terminology

Before we proceed further let's understand a few DRBD terms.

Resource: A resource is a set of things that form a replicated data set. It consists of a resource name, volumes, DRBD devices and a connection.

Resource name: Any arbitrary ASCII string excluding white space.

Volumes: Every resource consists of a set of volumes that share a single replication stream. Volumes contain the data set and some meta-data used by DRBD itself.

DRBD device: A virtual block device `/dev/drbdX`, corresponds to each volume in the resource, where *X* is the device's minor number.

Connection: A communication link between two nodes. Each resource will have exactly one connection.

Feature list

Though DRBD has matured into a system with a rich set of functionalities, let's look at some of the most important ones.

- Supports shared-secret authentication
- LVM (Logical Volume Manager) compatible
- Supports heartbeat/pacemaker resource agent integration
- Supports load balancing of read requests
- Automatic detection of the most up-to-date data after complete failure
- Delta resynchronisation
- Existing deployment can be configured with DRBD without losing data
- Automatic bandwidth management

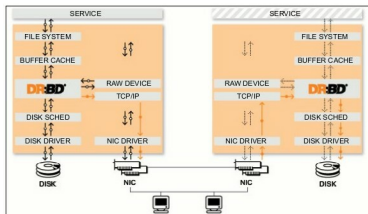


Figure 1: Position of DRBD within Linux

- Customisable tuning parameters
- Online data verification with peer

Components

DRBD functionalities are implemented in the following components:

drbd: This is a kernel module—the core—which actually does the work for us.

drbdadm: This is a userland administrative tool. It reads the configuration file and invokes the following tools with the required parameters, to get the job done.

drbdsetup: This is a userland administrative tool.

It configures the *drbd* kernel module. Usually, direct invocation by the user is not required. *drbdadmin* invokes whenever required.

drbdmeta: This userland administrative tool manipulates the meta data structure. Usually, direct invocation is not required.

Redundancy modes

DRBD can be set up in the following redundancy modes:

Single primary mode: In this mode, a DRBD *resource* takes the *primary* role on only one cluster member at a time. Since only the *primary* manipulates data, non-sharing file systems like ext3 and ext4 can be used in this configuration. This approach is used to set up a fail-over (HA) capable cluster.

Dual primary mode: In this mode, a DRBD *resource* takes the *primary* role on both the cluster members at a time. Since both nodes manipulate data, a sharing file system with distributed locking mechanisms like OCFS2 and GFC is required. This approach is used for load-balancing a cluster.

Replication modes

Different replication modes of DRBD are as follows:

Asynchronous (Protocol A): In this mode, a write operation on the *primary* is considered complete as soon as it is written on the local disk and is put in the TCP send buffer for replicating in peers. Data loss in the *secondary* may occur if the *secondary* fails before this write actually gets a chance to reach the *secondary* node disk. This is most often used in long distance replication set-ups where communication delay is a considerable factor.

Semi-synchronous (Protocol B): In this mode, a write operation on the *primary* is considered complete as soon as it is written on the local disk and the former has reached the peer node (not written to peer disk yet). Forced failure may not cause data loss, because the *primary* can detect that data has not reached a peer, so marks it as a failure and takes necessary action. But if the *secondary* fails after the data has reached it and before it is written to its local